

基于视觉扰乱与数字序列驱动的图片加密解密方法研究

摘要：随着数字图像在互联网中的广泛传播，图像信息安全问题日益突出。本文提出了一种基于视觉扰乱与数字序列驱动的图片加密解密方法，其核心思想是将原始图片分割为若干固定大小的小图块，按照预设的起点和顺序提取为一维子图片数组，再利用十进制数字序列（如圆周率 π 、自然常数 e 等）驱动的填充规则将子图片数组重新排列到空白网格中，形成视觉上完全混乱的加密图片。解密过程通过加密图片EXIF信息中存储的解密参数逆向恢复原始图片。该方法具有密钥空间大、实现简单、无需复杂变换运算等优点，适用于对图像内容进行轻量级保护。本文详细描述了加密解密算法的原理与实现，分析了其安全性，并给出了基于OpenCV和Pillow两种Python库的实现方案及Web交互式演示工具。

关键词：图片加密；视觉扰乱；数字序列；分块重排；EXIF元数据

1 引言

在信息技术高速发展的今天，数字图像已成为信息传播的重要载体。社交媒体、在线存储、医疗影像、军事侦察等领域大量依赖数字图像进行信息传递与记录。然而，图像在传输和存储过程中面临着被未授权访问、篡改和窃取的风险。因此，如何有效地保护数字图像的内容安全，成为信息安全领域的一个重要研究课题。

传统的图像加密方法主要分为两大类：一类是基于像素值变换的加密方法，如Arnold猫映射、混沌序列加密、DNA编码加密等，这类方法通过改变像素值或像素位置实现加密；另一类是基于压缩感知和频域变换的加密方法，如DCT域加密、小波变换加密等，这类方法在频域中对图像系数进行处理。这些方法各有优劣：像素值变换方法计算量小但密钥空间有限；频域变换方法安全性较高但计算复杂度大。

本文提出一种基于视觉扰乱与数字序列驱动的图片加密方法。该方法借鉴拼图（puzzle）的思想，将原始图片分割为若干固定大小的图块，通过多种提取起点和顺序将图块提取为一维序列，再利用十进制数字序列驱动的填充规则将图块散列到目标网格中。加密后的图片在视觉上呈现为完全打乱的混乱状态，而持有正确解密参数的用户可以精确还原原始图片。该方法不改变像素值，仅改变像素块的位置，具有实现简单、密钥空间大、可逆性良好等特点，为图像内容保护提供了一种轻量级解决方案。

2 算法原理

2.1 基本概念

设原始图片的宽度为 W ，高度为 H ，定义小图块（子图片单元）的大小为 $a \times b$ 像素，其中 a 为图块宽度， b 为图块高度。加密过程包含三个核心步骤：填充（Padding）、分块提取（Block Extraction）和数字序列驱动填充（Sequence-Driven Placement）。解密过程为加密过程的逆运算。

2.2 填充处理

由于原始图片的宽度和高度不一定能被 a 和 b 整除，需要对图片进行填充处理。填充规则如下：在图片的右侧添加黑色像素列，使图片总宽度 W' 满足 $W' = 0 \pmod{a}$ ，即 $W' = \text{ceil}(W/a) \times a$ ；在图片的下侧添加黑色像素行，使图片总高度 H' 满足 $H' = 0 \pmod{b}$ ，即 $H' = \text{ceil}(H/b) \times b$ 。填充后图片的尺寸为 $W' \times H'$ ，可以均匀地划分为 $m \times n$ 个 $a \times b$ 像素的子图块，其中 $m = H'/b$ （行数）， $n = W'/a$ （列数）。

348 px

Padding填充示意图（块大小4x3）



图1: Padding填充示意图。原始图片（346x266像素）在右侧添加2像素、下侧添加2像素的黑色区域，使填充后尺寸为348x268像素。

图1展示了Padding填充的具体过程。当块大小定义为4x3时，宽346像素需要填充2像素（348=4x87），高266像素需要填充2像素（268=4x67），填充后图片可被均匀划分为87x67个子图块。填充区域采用纯黑色（RGB值为0,0,0），以便在解密时精确裁剪。

2.3 分块提取

填充后的图片被划分为 $m \times n$ 个子图块，形成一个二维数组。分块提取的目的是按照指定的起点和顺序，将二维数组中的子图块逐一提取为一个长度为 $m \times n$ 的一维子图片数组A。

2.3.1 提取起点

提取起点决定了遍历的起始位置，共有四种选择：

起点编号	位置	起始单元格坐标
1	左上角	(1,1)
2	右上角	(1,n)
3	右下角	(m,n)
4	左下角	(m,1)

2.3.2 提取顺序

提取顺序定义了从起点开始遍历二维数组的路径规则，共有六种方式：

(1) **顺序1：顺时针螺旋** 从起点开始，按照顺时针方向从外向内螺旋式提取子图块。以左上角起点为例：首先沿第一行从左到右提取所有列，然后沿最后一列从上到下，再沿最后一行从右到左，最后沿第一列从下到上，完成最外圈提取后移至内侧次外圈起点继续，直到所有单元格提取完毕。

(2) **顺序2：逆时针螺旋** 与顺时针螺旋类似，采用从外向内的螺旋路径，但全程按逆时针方向提取。

(3) **顺序3：逐行前进** 按照行优先的顺序逐行提取子图块。当起点为其他角时，等效于对二维数组做相应旋转后从左上角开始提取。

(4) **顺序4：逐列前进** 按照列优先的顺序逐列提取子图块。

(5) **顺序5：逐行迂回前进** 按行提取，但相邻行方向交替，形成蛇形迂回路径。

(6) 顺序6：逐列迂回前进 按列提取，但相邻列方向交替，形成蛇形迂回路径。

六种提取顺序示意图（起点：左上角，5×6网格）

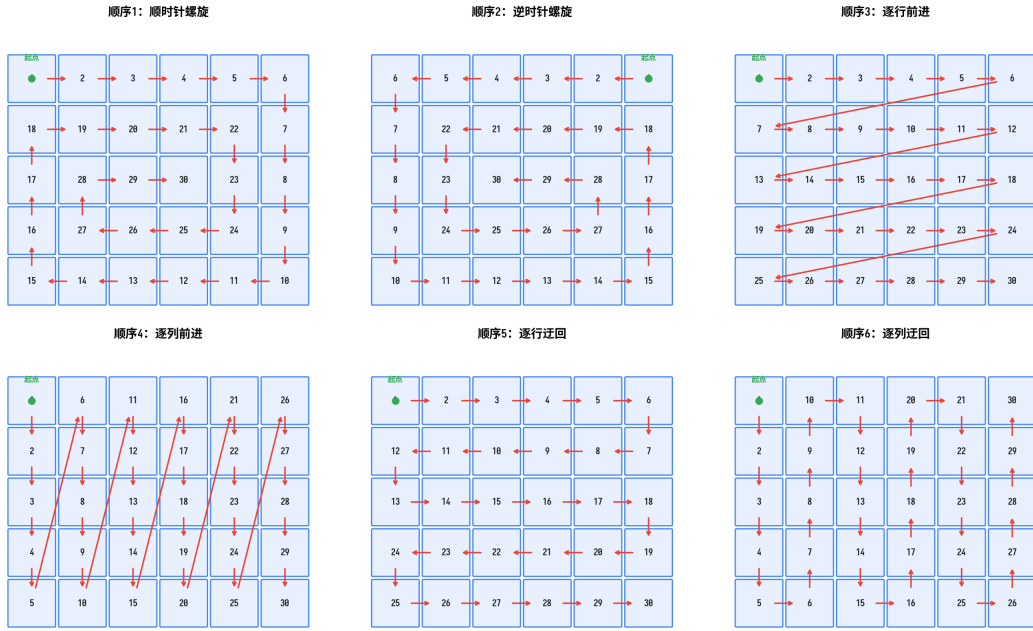


图2：六种提取顺序示意图（起点为左上角，5x6网格）

四种起点示意图（顺序：顺时针螺旋，5×6网格）

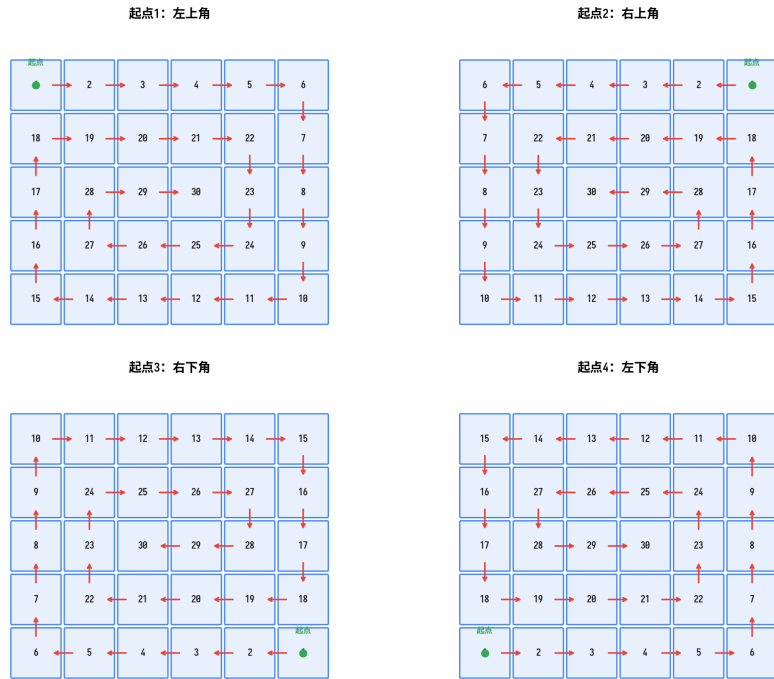


图3：四种起点示意图（顺时针螺旋，5x6网格）

2.4 加密与解密流程概览

在详细描述数字序列驱动ed的加扰填充之前，先给出完整的加密和解密流程图，以便读者从整体上把握算法的运作机制。

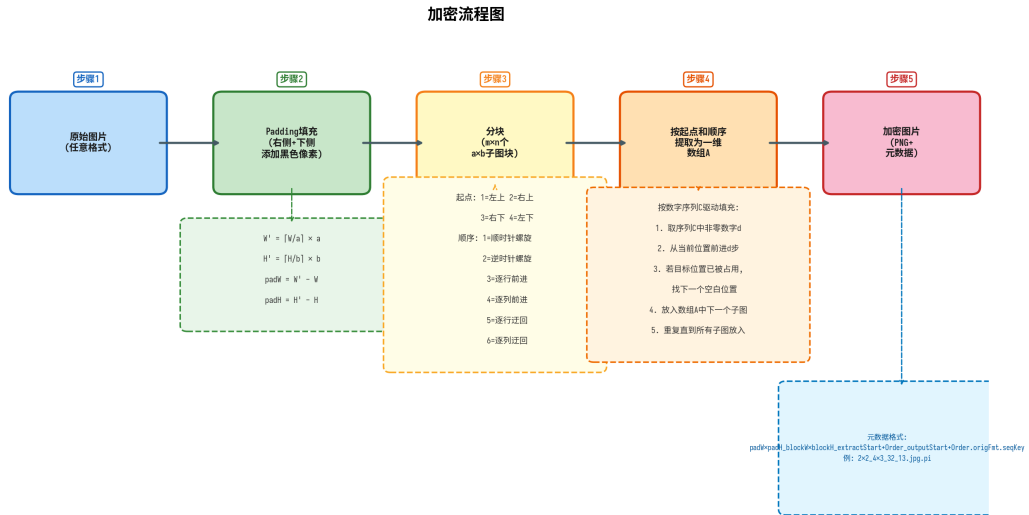


图4：加密流程图。原始图片输入 -> Padding填充 -> 分块 -> 按起点和顺序提取为一维数组A -> 数字序列C驱动填充生成加密图片 -> 写入PNG元数据。

加密流程包含五个核心步骤：第一步，读入原始图片；第二步，根据块大小a x b进行Padding填充；第三步，分割为m x n个子图块；第四步，按指定起点和顺序提取为一维数组A，再按数字序列C驱动的规则填充到空白网格B中；第五步，拼接为加密图片，保存为PNG格式并写入解密参数到元数据。

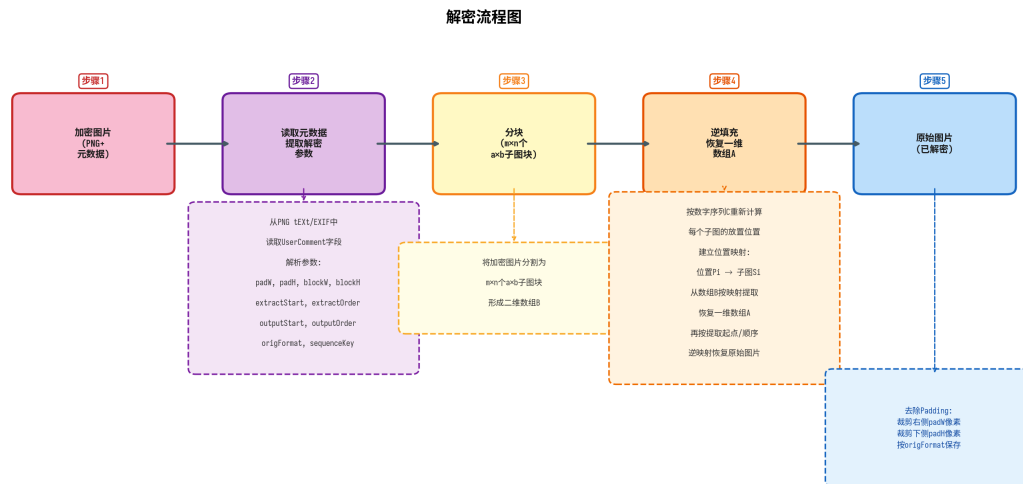


图5：解密流程图。读入加密图片 -> 提取元数据参数 -> 分块 -> 数字序列C逆填充恢复数组A -> 逆提取恢复原始图片 -> 去除Padding。

解密流程是加密流程的逆运算：读入加密PNG图片后，从元数据中提取解密参数（Padding尺寸、块大小、起点、顺序、数字序列标识），将加密图片分割为子图块形成二维数组B，利用数字序列C重新计算放置位置逆向恢复一维数组A，再逆映射回原始二维网格，最后裁剪Padding区域保存解密图片。

2.5 数字序列驱动的加扰填充

在获得一维子图片数组A后，按照数字序列驱动的规则将数组A中的子图片逐一填充到空白的m x n二维数组B中，形成加密图片。该步骤是整个加密算法的核心，通过十进制数字序列控制填充位置，实现子图片的散列化排列。

2.5.1 十进制数字序列

数字序列C是由十进制数字（0-9）组成的序列，推荐使用数学常数的十进制展开作为序列来源，例如：圆周率 $\pi = 3.141592653589793\dots$ ，自然常数 $e = 2.718281828459045\dots$ ，黄金比例 $\phi = 1.618033988749894\dots$ 。使用数学常数作为序列来源的优势在于：序列具有良好的伪随机性质，且接收方仅需指定常数代号（如" π "、" e "）即可在解密端重现相同序列。注意数字序列中必须包含足够多的非零数字（至少 $m \times n$ 个），因为数字0在填充规则中代表"不前进"（跳过）。

2.5.2 填充规则

- 填充过程按照指定的输出起点和输出顺序遍历空白的二维数组B，具体规则如下：
- 1. 初始化当前位置为输出起点，当前待填充的子图片索引为1。
 - 2. 从数字序列C中依次取一位数字d：若d=0则跳过；若d不为0，从当前位置沿输出顺序前进d步（循环遍历）。
 - 3. 到达目标位置后，若该位置为空则放入子图片；若已占用则沿输出顺序继续前进找到第一个空白位置。
 - 4. 放置完成后，从当前放置位置开始重复步骤2-3，放入下一个子图片。
 - 5. 当所有 $m \times n$ 个子图片全部放入数组B后，填充过程结束。

上述规则确保每个子图片被唯一放置且永不覆盖已放置的子图片。数字序列的非均匀性使得最终填充结果在视觉上呈现高度混乱状态。

2.5.3 填充示例

以4x4网格为例，输出起点为左上角，输出顺序为逐行前进，数字序列为圆周率 $\pi = 3.141592653589793\dots$ 。填充过程前几步如下：子图片1：d=3，从(1,1)前进3步到(1,4)；子图片2：d=1，从(1,4)前进1步到(2,1)；子图片3：d=4，从(2,1)前进4步到(3,1)；子图片4：d=1，从(3,1)前进1步到(3,2)；以此类推直到16个子图片全部放置完毕。

加密填充过程示例（4x4网格， $\pi=3.14159\dots$ ，起点：左上角，逐行前进）

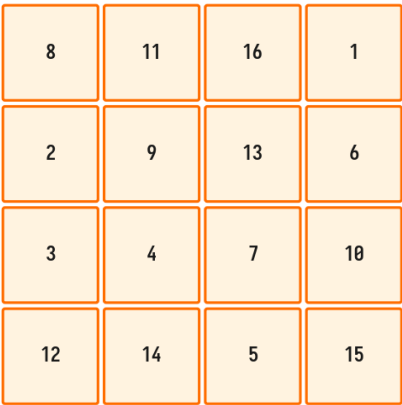


图6：加密填充过程示例（4x4网格， π 序列驱动）

2.6 加密图片输出

当所有子图片填充到数组B后，将 $m \times n$ 个子图片按其在数组B中的位置拼接为完整的加密图片，保存为PNG格式。选择PNG格式的原因是其支持无损压缩且支持EXIF元数据嵌入。解密参数保存到PNG图片EXIF区域的UserComment（标签0x9286）中，格式如下：

```
<宽度填充>x<高度填充>_<块宽>x<块高>_<提取起点><提取顺序>_<输出起点><输出顺序>.<原图格式>.<数字序列C>
```

字段	含义	示例
宽度填充像素数	水平方向右侧填充像素数	2
高度填充像素数	垂直方向下侧填充像素数	2
块宽/块高	子图块宽度a / 高度b	4x3

提取起点/顺序	分块提取时的起点和顺序编号	32
输出起点/顺序	填充时的输出起点和顺序编号	13
原图格式	原始图片的文件格式	jpg
数字序列C	数字序列标识	pi

若使用数学常数，该字段使用代号（pi写作"pi"，e写作"e"）；若使用自定义数字序列，则直接写入数字字符串。**示例：**原始图片a.jpg，宽高346x266，块4x3，水平填充2像素，垂直填充2像素，提取起点3，顺序2，输出起点1，顺序3，数字序列pi，则解密参数为：

```
2x2_4x3_32_13.jpg.pi
```

2.7 解密算法

解密是加密的逆过程，包括以下步骤：

- 1. **提取解密参数：**从加密PNG图片的EXIF区域读取解密参数字符串，解析出各字段值。
- 2. **分块提取：**将加密图片按块大小a x b分割为m x n个子图块，形成二维数组B。
- 3. **逆向恢复数组A：**根据数字序列C、输出起点和输出顺序，重新计算每个子图片的放置位置，建立位置映射关系，从数组B中逆向恢复一维数组A。
- 4. **逆提取恢复图片：**根据提取起点和顺序的逆运算，将数组A中的子图片重排回原始二维网格。
- 5. **去除Padding：**裁剪图片右侧和下侧的黑色填充区域，恢复原始尺寸。
- 6. **保存解密图片：**按原始图片格式保存。

3 算法安全性分析

3.1 密钥空间分析

本算法的密钥空间由以下参数共同决定：**块大小(a, b)**决定了图片的分割粒度，以8x8块为例，256x256图片产生1024个子图块；**提取起点**（4种）和**提取顺序**（6种）共有24种组合；**输出起点**（4种）和**输出顺序**（6种）同样有24种组合；**数字序列C**是密钥空间的主要贡献者，若使用自定义序列则理论密钥空间为无穷大。综合考虑，即使仅考虑起点和顺序组合，密钥空间已达24 x 24 = 576种；加上块大小和数字序列变化，穷举攻击代价极高。

3.2 抗攻击分析

- (1) **抗穷举攻击** 由于数字序列C的不确定性，攻击者无法通过穷举起点和顺序组合来破解。填充过程的非线性（跳过0、避免覆盖）使得反推填充参数的计算复杂度远高于简单排列组合。
- (2) **抗统计分析** 加密图片中相邻子图块来源于原图不相邻区域，打破了像素空间相关性，攻击者无法通过局部统计特征推断原图内容。
- (3) **抗已知明文攻击** 即使拥有明文-密文对，由于填充过程的非线性，逆向推导数字序列在数学上困难。

3.3 安全性局限

本方法属于位置置乱类加密算法，不改变像素值，存在以下局限：若攻击者同时获取加密图片和部分解密参数，搜索空间将大幅缩减；对于明显视觉特征的图片，部分内容仍可能被肉眼识别；建议与像素值变换加密方法组合使用以增强安全性。

4 实现方案

4.1 OpenCV版本实现

基于OpenCV库的实现方案利用cv2模块进行图像读写与像素操作，使用piexif库处理EXIF元数据。核心步骤包括：(1) cv2.imread()读取原始图片；(2) cv2.copyMakeBorder()添加黑色填充；(3) NumPy数组切片提取子图块；(4) 实现六种遍历顺序生成函数；(5) 实现数字序列驱动填充函数；(6) piexif库写入EXIF解密参数；(7)

解密时从EXIF读取参数逆向恢复。

4.2 Pillow版本实现

基于Pillow库的实现方案利用PIL.Image模块进行图像操作，核心逻辑与OpenCV版本相同，主要差异在于图像表示方式：OpenCV使用NumPy数组（BGR格式），Pillow使用Image对象（RGB格式）；像素访问方式不同：OpenCV通过数组索引，Pillow通过crop()和paste()方法；填充方式不同：OpenCV使用copyMakeBorder()，Pillow使用Image.new()创建新画布并粘贴。

4.3 Web交互式工具

基于HTML+JavaScript实现了交互式加密解密工具，用户可在浏览器中选择本地图片并预览、配置加密参数、执行加密操作并预览加密图片、保存加密图片（含EXIF解密参数）到本地、加载加密图片并执行解密操作。该工具采用纯前端实现，所有运算在浏览器端完成，无需服务器支持，保障了用户图片数据的隐私安全。

5 实验结果与分析

5.1 加密效果

对多种类型的测试图片进行加密实验（风景照片、人像照片、文字截图和几何图形），实验参数：块大小 8×8 ，提取起点左上角（1），提取顺序逆时针螺旋（2），输出起点右下角（3），输出顺序逐列迂回（6），数字序列 π 。实验结果表明：无论何种类型的原始图片，加密后均呈现高度混乱状态，无法辨识原图内容。风景照片色彩区域被打散为随机色块；人像面部特征完全消失；文字字符被打乱为不可读碎片；几何图形规律性结构被彻底破坏。

5.2 解密保真度

由于本方法仅改变子图块位置而不修改像素值，解密后图片与原始图片在像素级别完全一致（无损加密）。唯一差异来源于Padding区域：解密后裁剪填充区域即恢复原始尺寸，只要Padding像素数正确，裁剪后图片与原图完全相同。

5.3 性能分析

加密和解密的时间复杂度主要由分块提取 $O(m \times n)$ 和数字序列驱动填充 $O(m \times n)$ 两部分组成，总时间复杂度为 $O(m \times n)$ ，与图片尺寸和块大小呈线性关系。实际测试中，对 1920×1080 像素图片使用 8×8 块大小（32400个子图块），加密和解密操作均在1秒内完成，表明本方法具有良好的实时性能。

6 结论

本文提出了一种基于视觉扰乱与数字序列驱动的图片加密解密方法。该方法通过分块提取和数字序列驱动的填充规则，将原始图片转换为视觉上完全混乱的加密图片，解密过程通过存储在EXIF中的参数逆向还原。该方法具有以下特点：**1. 轻量级**：仅涉及图像分块和位置重排，计算效率高；**2. 密钥空间大**：多种参数组合提供丰富密钥空间；**3. 无损加密**：仅改变像素位置，解密后像素级一致；**4. 自包含**：解密参数嵌入EXIF区域，无需额外密钥传输通道；**5. 灵活可扩展**：支持任意十进制数字序列作为驱动序列。

未来的研究方向包括：将本方法与像素值变换加密方法结合，增强抗分析能力；引入动态块大小策略，进一步提升安全性；以及在GPU上并行化实现，提高大尺寸图片的处理速度。

参考文献

- [1] Fridrich J. Symmetric ciphers based on two-dimensional chaotic maps[J]. International Journal of Bifurcation and Chaos, 1998, 8(06): 1259-1284.
- [2] Arnold V I, Avez A. Ergodic problems of classical mechanics[M]. Benjamin, 1968.
- [3] Matthews R. On the derivation of a "chaotic" encryption algorithm[J]. Cryptologia, 1989, 13(1): 29-42.
- [4] 陈师, 孙克辉, 牟俊. 基于混沌系统的图像加密算法研究综述[J]. 计算机工程与应用, 2021, 57(10): 31-43.

- [5] Wang X, Teng L, Qin X. A novel colour image encryption algorithm based on chaos[J]. Signal Processing, 2012, 92(4): 1101-1108.